CLIENT–SERVER ARCHITECTURE IN WEB APPLICATIONS

How a browser sends a request, how the server processes it, and how the response returns.

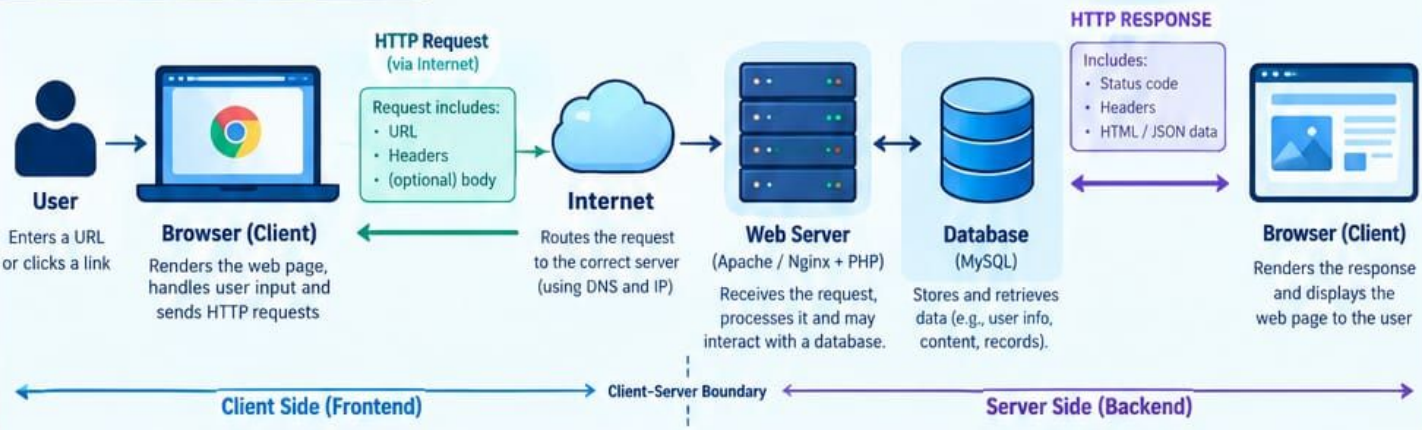
ITUE203 – Web Development Frameworks | **Unit 1** – Web technologies, protocols and web application architecture

Student 1:
Joshi Tarana
Enrollment No.: 25DCE034

Student 2:
Hetvi Kalkani
Enrollment No.: 25DCE040

3rd Semester | CHARUSAT

1. OVERVIEW: THE BIG PICTURE



2. STEP-BY-STEP FLOW

1. **User Enters a URL / Action**
   The browser takes the input and checks its cache or DNS to find the server's IP address.
2. **DNS Resolution**
   Domain name is converted to an IP address (e.g., charusat.ac.in → 192.168.1.1).
3. **HTTP Request is Sent**
   The browser sends an HTTP/HTTPS request to the server using the correct IP and port (e.g., port 443 for HTTPS).
4. **Server Processes the Request**
   The web server receives the request, runs the required logic (e.g., PHP), and may query the database.
5. **Database Interaction**
   If needed, the server fetches or updates data in the database and gets the result.
6. **Response is Sent Back**
   The server sends an HTTP response with status code, headers and data (HTML/JSON).
7. **Browser Renders the Page**
   The browser interprets the response and displays the web page or updates the UI.

3. KEY COMPONENTS

Client (Browser)
- Handles user interface
- Manages cookies and cache
- Sends HTTP/HTTPS requests

Web Server
- Listens for incoming requests
- Serves static files (HTML, CSS, JS)
- Runs server-side code (PHP)

Application Layer (PHP)
- Processes business logic
- Validates input
- Interacts with the database

Database (MySQL)
- Stores application data
- Executes queries
- Returns data to the server

4. PROTOCOLS & TECHNOLOGIES

- **HTTP/HTTPS** — Protocol for client-server communication. HTTPS adds encryption (TLS).
- **DNS** — Resolves domain names to IP addresses.
- **TCP/IP** — Handles data transmission over the network.
- **HTML** — Structures the content.
- **CSS** — Styles the web page.
- **JavaScript** — Adds interactivity on the client side.
- **PHP** — Server-side scripting language.
- **MySQL** — Database management system.

5. REQUEST & RESPONSE EXAMPLE

HTTP Request (simplified)

```
GET /students HTTP/1.1
Host: charusat.ac.in
User-Agent: Mozilla/5.0
Accept: text/html
Cookie: session_id=abc123
```

HTTP Response (simplified)

```
HTTP/1.1 200 OK
Content-Type: text/html
Set-Cookie: session_id=xyz789

<html>
  <body> ... </body>
</html>
```

6. WHY IT MATTERS?

- Enables web applications to work over the internet.
- Provides a clear separation between client and server responsibilities.
- Allows scalability (easy to add more servers).
- Supports modern technologies like cloud and APIs.

7. REAL-WORLD EXAMPLE



Example: When you open the CHARUSAT website, your browser sends an HTTPS request to the server. The server processes it and returns the webpage.

8. KEY TAKEAWAYS

- Browser sends HTTP/HTTPS requests to the server.
- Server processes the request, may use a database, and sends back a response.
- HTTPS ensures data is encrypted.
- Each layer has a specific role.
- This architecture powers modern web applications and websites.

Better Communication | Secure Data | Powerful Web Applications

ITUE203 – CHARUSAT